Laporan Praktikum Pemrograman Berorientasi Obyek (PBO) – Modul 11

Collection dan Pengelolaan Kumpulan Objek (Sistem Kasir Sederhana Berbasis Agile)

Nama : Rafael Rizky Luga Tanjung **Dosen** : Nirwana Haidar Hari, S.Pd., M.Kom.
NRP : 3125522007 **Kampus**: PENS PSDKU Sumenep (2026)

BAGIAN 1 – KONSEPSI AGILE, COLLECTION FRAMEWORK DAN ENKAPSULASI PENGELOLA OBJEK

1. Latar Belakang & Analisis Agile Project Kickoff (Sprint P11): Melanjutkan evolusi proyek Sistem Kasir Sederhana dari Pertemuan 10 dengan kerangka kerja Agile (Scrum). Fokus Sprint P11 adalah Epic: *Dynamic Catalog Inventory, Generics Collection & Strict Encapsulation*. Pada P10, objek produk masih diinisialisasi secara statis dan terisolasi di method driver tanpa wadah dinamis. Pada Sprint P11, arsitektur sistem ditingkatkan dengan memperkenalkan class pengelola terpusat (KatalogProduk) yang memanfaatkan Java Collections Framework (List<Produk> diimplementasikan dengan ArrayList). Sistem kini mendukung manajemen siklus hidup barang (tambah, cari, hapus, tampilkan) dengan penegakan kode produk unik dan enkapsulasi ketat sehingga memori internal toko terlindungi dari modifikasi liar.

• **Agile User Stories & Acceptance Criteria:** (1) *Sebagai Kasir*, saya ingin sistem mengelola seluruh daftar produk toko dalam satu katalog dinamis, sehingga saya dapat memeriksa ketersediaan barang secara terpusat. (2) *Sebagai Kasir*, saya ingin mencari produk berdasarkan kode unik maupun potongan nama barang. (3) *Sebagai Pemilik Toko*, saya ingin sistem menolak pendaftaran produk baru jika kode produk sudah pernah terdaftar, sehingga tidak terjadi duplikasi data inventaris. (4) *Sebagai Pemilik Toko*, saya ingin koleksi produk internal dienkapsulasi rapat sehingga class lain tidak dapat menghapus atau memanipulasi list secara ilegal.

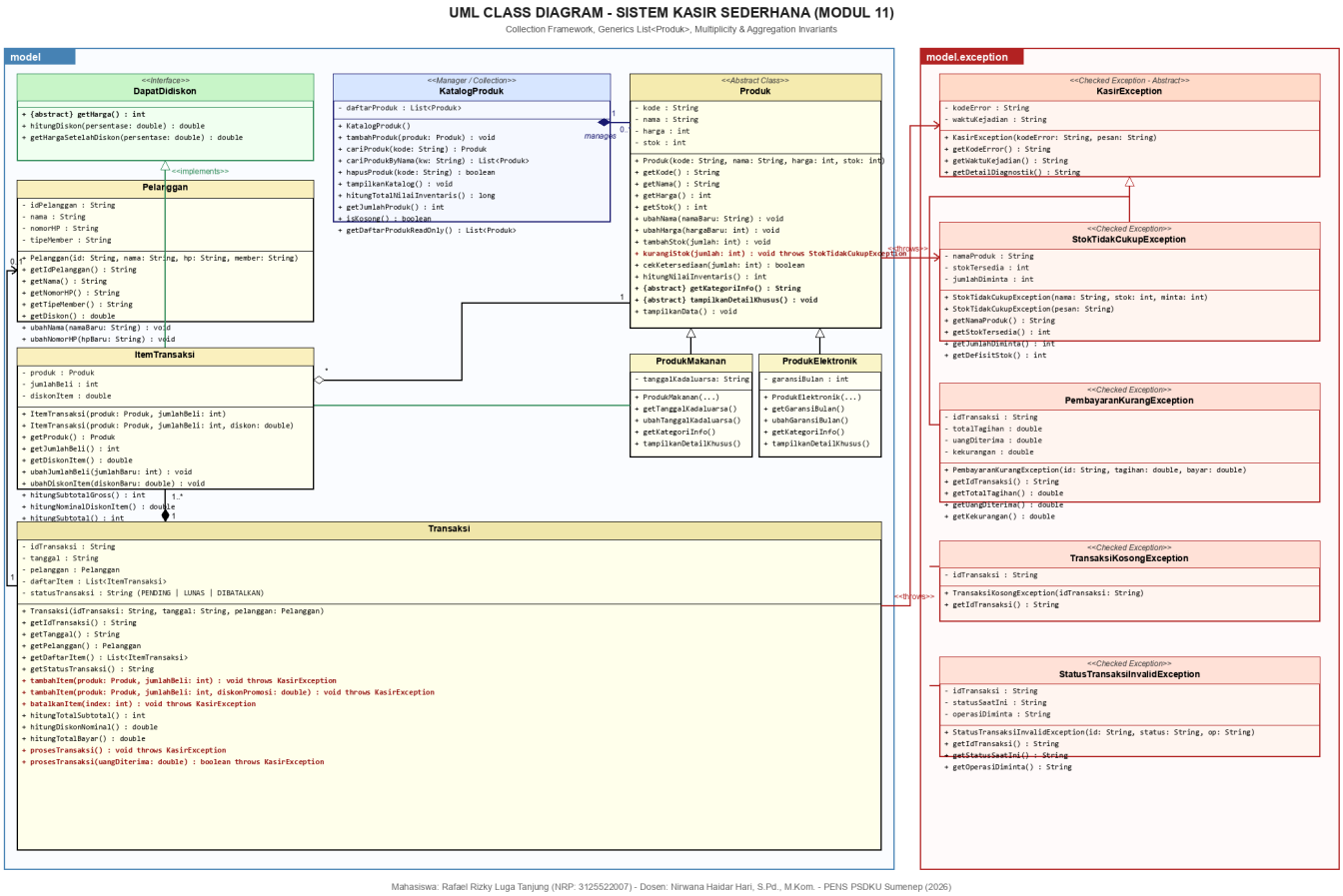
2. Desain Operasi Collection pada Class KatalogProduk (Domain POS):

Operasi / Fitur	Method Java & Signature	Validasi & Aturan Bisnis	Hasil & State Akhir
Tambah Produk	tambahProduk(Produk p)	Cek null dan keunikan kode via cariProduk()	Menolak kode duplikat (throw IllegalArgumentException). Jika valid, tersimpan di list.
Cari by Kode Unik	cariProduk(String kode)	Perulangan for-each mencocokkan kode secara case-insensitive	Mengembalikan referensi objek Produk konkret jika ada, atau null jika tidak terdaftar.
Cari by Nama	cariProdukByNama(String kw)	Pencarian fleksibel berbasis substring lowercase	Mengembalikan List<Produk> baru berisi semua produk yang cocok.
Hapus Produk	hapusProduk(String kode)	Mencari objek berdasarkan kode sebelum dieksekusi	Menghasilkan true & menghapus item jika ada, atau false jika kode tidak ditemukan.
Tampilkan Katalog	tampilkanKatalog()	Pemeriksaan isEmpty() dan pemformatan tabel polimorfis	Mencetak seluruh rincian barang, kategori (Makanan/Elektronik), dan total aset inventaris.
Proteksi Enkapsulasi	getDaftarProdukReadOnly()	Membungkus list dengan Collections.unmodifiableList()	Mencegah pemanggilan langsung method mutator (seperti .clear() atau .remove()) dari luar class.

3. Prinsip Enkapsulasi List & Pencegahan Kebocoran State: Sesuai mandat Modul 11, variabel daftarProduk dideklarasikan sebagai private final List<Produk>. Penggunaan modifier final menjamin referensi instance list tidak dapat diganti dengan objek list baru di runtime, sedangkan private memastikan modifikasi hanya dapat dilakukan melalui method resmi class (seperti tambahProduk dan hapusProduk). Jika class luar membutuhkan pembacaan daftar produk, sistem menyediakan getDaftarProdukReadOnly() yang mengembalikan unmodifiable wrapper, sehingga operasi perusakan seperti list.clear() akan seketika melempar UnsupportedOperationException.

BAGIAN 2 – ARSITEKTUR KODE, UML CLASS DIAGRAM DAN TABEL PEMETAAN JAVA

4. UML Class Diagram Terbaru (Modul 11 - Collection & Manager Architecture):



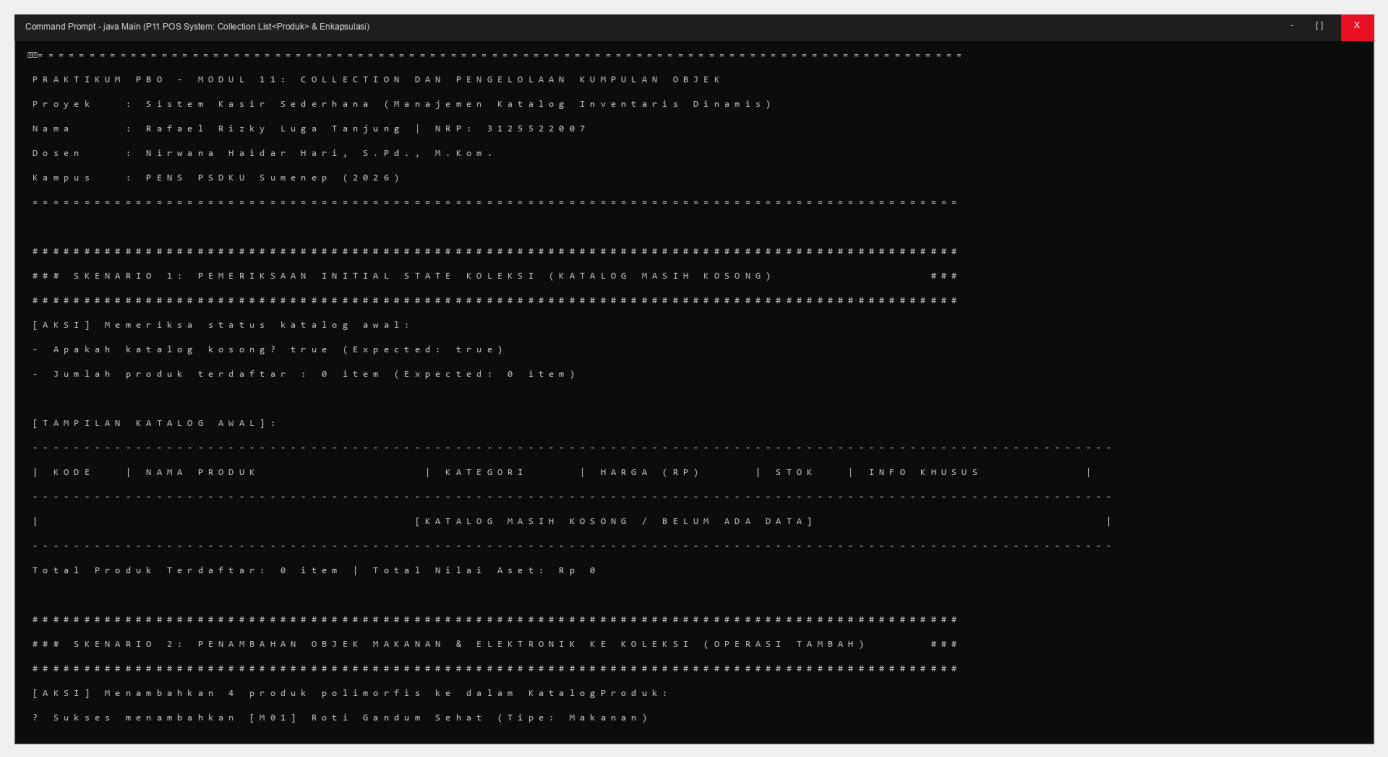
*Gambar 1. UML Class Diagram Sistem Kasir Sederhana Modul 11 (Relasi Komposisi KatalogProduk 1 *-- 0..* Produk)*

5. Tabel Pemetaan Elemen UML & Collection Framework ke Source Code Java:

Elemen UML / Konsep	Notasi di Diagram UML	Implementasi pada Source Code Java
Class Pengelola Koleksi	class KatalogProduk <>	public class KatalogProduk (mengelola siklus hidup List<Produk>)
Generics Collection List	- daftarProduk : List<Produk>	private final List<Produk> daftarProduk = new ArrayList<>();
Relasi Komposisi	KatalogProduk "1" *-- "0..*" Produk	KatalogProduk memiliki kepemilikan penuh atas instansiasi objek-objek Produk.
Polimorfisme Koleksi	Produk <|-- ProdukMakanan / Elektronik	List<Produk> dapat menampung ProdukMakanan dan ProdukElektronik secara seragam.
Enkapsulasi Read-Only	+ getDaftarProdukReadOnly() : List<Produk>	return Collections.unmodifiableList(daftarProduk); (menolak mutasi luar)
Integrasi Transaksi	ItemTransaksi "*" o-- "1" Produk	Transaksi mengambil objek Produk dari KatalogProduk lalu mengaitkannya ke keranjang.

BAGIAN 3 – EKSEKUSI PROGRAM, SKENARIO UJI DAN EVALUASI MODUL 11

6. Bukti Eksekusi Terminal (Main.java & MainLab.java Bebas Error):



Gambar 2. Bukti Running Program Main.java Sistem Kasir Sederhana (8 Skenario Collection Teruji 100% Sukses)



Gambar 3. Bukti Eksekusi Praktikum Dasar Modul 11 (MainLab.java: Kasus Koleksi Perpustakaan)

7. Ringkasan Hasil Pengujian Komprehensif (Tabel Skenario Pengujian Modul 11):

No	Fokus Skenario Uji	Aksi / Input Pengujian	Hasil yang Diharapkan	Jumlah Data Akhir	Status
1	Initial State Kosong	Periksa katalog baru dibuat sebelum ada data	isKosong()=true, tampilkan pesan katalog kosong	0 item	**SUKSES**
2	Penambahan Polimorfis	Tambah M01, M02 (Makanan) & E01, E02 (Elektronik)	Keempat produk tersimpan sukses dalam List<Produk>	4 item	**SUKSES**
3	Pencegahan Duplikat	Tambah produk baru dengan kode terdaftar 'M01'	Throw IllegalArgumentException, penambahan ditolak	4 item	**SUKSES**
4	Pencarian Kode & Nama	Cari kode 'E01', kode 'X99', dan nama 'Mechanical'	'E01' ketemu, 'X99' return null, nama hasilkan 1 item	4 item	**SUKSES**
5	Operasi Penghapusan	Hapus kode terdaftar 'M02' dan kode fiktif 'E99'	Hapus M02 return true, hapus E99 return false	3 item	**SUKSES**

6	Integrasi Transaksi POS	Ambil M01 & E01 dari katalog, bayar Rp 200.000	Checkout sukses, stok di dalam katalog ikut terpotong	3 item	**SUKSES**
7	Uji Pertahanan Exception	Beli 50 unit Keyboard Mechanical (stok hanya 8)	Throw StokTidakCukupException, stok katalog aman 8 unit	3 item	**SUKSES**
8	Uji Proteksi Enkapsulasi	Panggil getDaftarProdukReadOnly().clear()	Throw UnsupportedOperationException, modifikasi liar ditolak	3 item	**SUKSES**

8. Jawaban Pertanyaan Evaluasi Modul 11:

1. Apa perbedaan Buku (atau Produk) dan List<Buku> (List<Produk>)?

Produk merepresentasikan satu entitas objek tunggal yang memiliki atribut konkret (kode, nama, harga, stok) dan logika bisnis individual. Sedangkan List<Produk> merupakan struktur data koleksi (wadah dinamis) di memori RAM yang dapat menampung banyak referensi objek Produk secara terorganisir, serta menyediakan fungsionalitas untuk menambah, mencari, mengiterasi, dan menghapus elemen-elemen tersebut secara terpusat.

2. Mengapa indeks list kurang tepat dijadikan identitas tetap sebuah buku atau produk?

Karena indeks pada ArrayList bersifat dinamis dan posisinya relatif terhadap elemen lain. Jika salah satu elemen di awal atau di tengah dihapus (misalnya elemen indeks ke-0 dihapus), maka seluruh elemen di belakangnya akan otomatis bergeser maju mengisi kekosongan (elemen indeks ke-1 menjadi indeks ke-0). Jika sistem kasir mengandalkan indeks sebagai identitas transaksi, maka transaksi berikutnya akan salah mereferensikan barang. Oleh karena itu, identitas barang harus menggunakan atribut unik domain yang permanen dan tidak terpengaruh oleh pergeseran memori, yaitu **Kode Produk**.

3. Mengapa pengecekan kode duplikat dilakukan sebelum add()?

Untuk menegakkan prinsip *Fail-Safe Invariants* dan integritas data koleksi. Jika objek dimasukkan terlebih dahulu ke dalam list lalu baru divalidasi, maka memori internal sudah terlanjur tercemar oleh data duplikat. Apabila terjadi kegagalan atau pembatalan, sistem harus melakukan operasi rollback manual yang rawan bug. Dengan melakukan pemeriksaan pra-kondisi (*pre-condition check*) di awal melalui pencarian kode, operasi yang melanggar aturan bisnis langsung digagalkan tanpa pernah menyentuh atau mengubah isi koleksi.

4. Apa risiko jika collection internal dapat diubah langsung oleh class lain?

Hal ini merusak prinsip fundamental **Enkapsulasi** (*Information Hiding*). Class eksternal dapat secara bebas memanipulasi elemen tanpa melewati aturan bisnis (misalnya mengosongkan list dengan .clear(), menyisipkan objek null, atau memasukkan produk dengan kode duplikat). Hal ini mengakibatkan *bypassing* validasi yang telah dibangun pada class pengelola, merusak konsistensi stok inventaris toko, menyulitkan pelacakan bug, dan berpotensi memicu kegagalan fatal seperti NullPointerException pada alur transaksi kasir.